

Rest Parameters and Spread Syntax - Simple Hinglish Summary

Source: <https://javascript.info/rest-parameters-spread>

Main baat

JS ke kai built-in functions **kitne bhi arguments** le sakte hain. Jaise:

- `Math.max(arg1, arg2, ..., argN)`: sabse bada return karta hai
- `Object.assign(dest, src1, ..., srcN)`: `src1..N` ki properties `dest` me copy karta hai

Is chapter me seekhenge ki **aise functions khud kaise banayein**, aur **arrays ko aise functions me kaise pass karein**.

1. Rest parameters `...`

Function ko **kitne bhi arguments** ke saath call kar sakte hain, chahe wo kaise bhi define ho:

```
javascript

function sum(a, b) {
  return a + b;
}

alert( sum(1, 2, 3, 4, 5) );
```

"Zyada" arguments ki wajah se error nahi aata. Lekin result me sirf pehle do gine jate hain, isliye upar ka result `3` hai.

Baaki parameters ko function definition me **teen dots** `...` aur ek array ke naam se le sakte hain. Dots ka matlab literally: "**bachhe hue saare parameters ko ek array me jama karo.**"

```
function sumAll(...args) { // args array ka naam hai
  let sum = 0;

  for (let arg of args) sum += arg;

  return sum;
}

alert( sumAll(1) );      // 1
alert( sumAll(1, 2) );  // 3
alert( sumAll(1, 2, 3) ); // 6
```

Pehle kuch parameters variables me aur baaki array me bhi le sakte hain:

javascript

```
function showName(firstName, lastName, ...titles) {
  alert( firstName + ' ' + lastName ); // Julius Caesar

  // baaki titles array me jate hain
  // yaani titles = ["Consul", "Imperator"]
  alert( titles[0] );    // Consul
  alert( titles[1] );    // Imperator
  alert( titles.length ); // 2
}

showName("Julius", "Caesar", "Consul", "Imperator");
```

Rest parameters hamesha aakhir me hone chahiye. Wo saare bache hue arguments ikatthe karte hain, isliye ye galat hai aur error deta hai:

javascript

```
function f(arg1, ...rest, arg2) { // ...rest ke baad arg2 ?!
  // error
}
```

2. **arguments** variable

Ek special array-like object **arguments** hota hai jisme saare arguments index ke hisaab se hote hain:

javascript

```
function showName() {
  alert( arguments.length );
  alert( arguments[0] );
  alert( arguments[1] );

  // ye iterable hai
  // for(let arg of arguments) alert(arg);
}

// dikhata hai: 2, Julius, Caesar
showName("Julius", "Caesar");

// dikhata hai: 1, Ilya, undefined (dusra argument nahi hai)
showName("Ilya");
```

Purane zamane me rest parameters language me nahi the, aur saare arguments pane ka `arguments` **hi ekmatra tarika** tha. Ab bhi chalta hai, purane code me milta hai.

Nuksan: `arguments` array-like aur iterable hai, lekin **array nahi hai**. Isme array methods nahi chalte, jaise `arguments.map(...)` nahi kar sakte. Aur ye **hamesha saare arguments** rakhta hai, rest parameters ki tarah hissa-hissa nahi le sakte.

Isliye in features ki zarurat ho to **rest parameters** preferred hain.

Arrow functions me `arguments` nahi hota. Arrow function me `arguments` access karo to wo bahar ke "normal" function se liya jata hai:

```
javascript

function f() {
  let showArg = () => alert(arguments[0]);
  showArg();
}

f(1); // 1
```

Yaad rakho, arrow functions ka apna `this` nahi hota. Ab pata chala ki unka special `arguments` object bhi nahi hota.

3. Spread syntax

Abhi humne dekha ki parameters ki list se **array** kaise banate hain. Lekin kabhi **ulta** karna padta hai.

Jaise built-in `Math.max` numbers ki list me se sabse bada deta hai:

javascript

```
alert( Math.max(3, 5, 1) ); // 5
```

Ab maan lo humare paas array `[3, 5, 1]` hai. `Math.max` ko isse kaise call karein?

Array ko "jaisa hai waisa" pass karna **kaam nahi karta**, kyunki `Math.max` ko numeric arguments ki list chahiye, ek array nahi:

javascript

```
let arr = [3, 5, 1];  
  
alert( Math.max(arr) ); // NaN
```

Aur haath se `Math.max(arr[0], arr[1], arr[2])` bhi nahi likh sakte, kyunki pata nahi kitne elements honge.

Spread syntax madad karta hai! Dikhne me rest parameters jaisa `(...)`, lekin **bilkul ulta** kaam karta hai.

Function call me `...arr` likhne par wo iterable `arr` ko **arguments ki list me "expand"** kar deta hai:

javascript

```
let arr = [3, 5, 1];  
  
alert( Math.max(...arr) ); // 5 (spread array ko arguments ki list banata hai)
```

Kai iterables ek saath:

javascript

```
let arr1 = [1, -2, 3, 4];  
let arr2 = [8, 3, -8, 1];  
  
alert( Math.max(...arr1, ...arr2) ); // 8
```

Normal values ke saath mix:

javascript

```
alert( Math.max(1, ...arr1, 2, ...arr2, 25) ); // 25
```

Arrays merge karne ke liye:

```
javascript

let arr = [3, 5, 1];
let arr2 = [8, 9, 15];

let merged = [0, ...arr, 2, ...arr2];

alert(merged); // 0,3,5,1,2,8,9,15 (0, phir arr, phir 2, phir arr2)
```

Upar arrays se dikhaya, lekin **koi bhi iterable** chalta hai. Jaise string ko characters ke array me badalna:

```
javascript

let str = "Hello";

alert( [...str] ); // H,e,l,l,o
```

Spread andar se **iterators** use karta hai, bilkul `for..of` ki tarah. Isliye string ke liye `for..of` characters deta hai aur `...str` `"H", "e", "l", "l", "o"` ban jata hai.

Is kaam ke liye `Array.from` bhi chalta hai:

```
javascript

let str = "Hello";

alert( Array.from(str) ); // H,e,l,l,o
```

Result wahi hai, `[...str]` jaisa.

Lekin `Array.from(obj)` aur `[...obj]` me chhota farak hai:

- `Array.from` array-likes aur iterables **dono** par kaam karta hai.
- **Spread syntax** sirf **iterables** par kaam karta hai.

Isliye kisi cheez ko array banane ke kaam me `Array.from` aksar zyada universal hota hai.

4. Array/object ki copy banana

Yaad hai pehle `Object.assign()` dekha tha? Wahi kaam spread syntax se bhi ho sakta hai.

Array copy:

javascript

```
let arr = [1, 2, 3];

let arrCopy = [...arr]; // array ko parameters ki list me spread karo
                        // phir result ko naye array me daalo

// kya arrays ka content same hai?
alert(JSON.stringify(arr) === JSON.stringify(arrCopy)); // true

// kya arrays barabar hain?
alert(arr === arrCopy); // false (same reference nahi)

// original array badalne se copy nahi badalti:
arr.push(4);
alert(arr);           // 1, 2, 3, 4
alert(arrCopy);       // 1, 2, 3
```

Object ki copy bhi aise hi:

javascript

```
let obj = { a: 1, b: 2, c: 3 };

let objCopy = { ...obj }; // object ko spread karo
                        // result naye object me

// kya objects ka content same hai?
alert(JSON.stringify(obj) === JSON.stringify(objCopy)); // true

// kya objects barabar hain?
alert(obj === objCopy); // false (same reference nahi)

// original object badalne se copy nahi badalti:
obj.d = 4;
alert(JSON.stringify(obj));           // {"a":1,"b":2,"c":3,"d":4}
alert(JSON.stringify(objCopy));       // {"a":1,"b":2,"c":3}
```

Ye tarika `Object.assign({}, obj)` (ya array ke liye `Object.assign([], arr)`) se **bahut chhota** hai, isliye jab bhi ho sake ise use karna behtar hai.

Dhyan: ye bhi **shallow copy** hai (nested objects reference se copy hote hain).

Summary

Code me `"..."` dikhe to wo ya to **rest parameters** hain ya **spread syntax**. Fark pehchanne ka aasan tarika:

- `...` **function parameters ke end me** ho to ye **rest parameters** hain: bache hue arguments ki list ko **array me jama** karte hain.
- `...` **function call (ya aisi jagah)** me ho to ye **spread syntax** hai: array ko **list me expand** karta hai.

Use patterns:

- Rest parameters:** aise functions banane ke liye jo **kitne bhi arguments** lein.
- Spread syntax:** aise functions me **array pass** karne ke liye jinhe aam taur par bahut saare arguments ki **list** chahiye.

Dono saath me list aur array ke beech aasani se "safar" karne me madad karte hain.

Function call ke saare arguments "**purane tarike**" `arguments` me bhi milte hain: array-like iterable object.

Quick Summary

	Rest parameters	Spread syntax
Kahan	Function definition ke end me	Function call me ya array/object literal me
Kya karta hai	List → array (jama karta hai)	Array → list (failata hai)
Example	<code>function f(a, ...rest) {}</code>	<code>Math.max(...arr)</code>

Kaam	Kaise
Kitne bhi arguments lena	<code>function sumAll(...args)</code>
Pehle kuch alag, baaki ek array	<code>function f(first, second, ...rest)</code>
Array ko arguments ki list banana	<code>Math.max(...arr)</code>
Arrays merge	<code>[0, ...arr1, 2, ...arr2]</code>
String → characters	<code>[...str]</code> ya <code>Array.from(str)</code>
Array copy	<code>let copy = [...arr]</code>
Object copy	<code>let copy = { ...obj }</code>
Purana tarika	<code>arguments</code> (array-like, array nahi)

Yaad rakho:

- `...rest` hamesha **aakhri parameter** hona chahiye.
- `arguments` array nahi hai (`map`), (`filter`) nahi chalte), aur **arrow functions me nahi hota**.
- Spread sirf **iterables** par chalta hai, (`Array.from`) array-likes par bhi.
- `[...arr]` aur `{...obj}` **shallow copy** banate hain.
- Jo `Object.assign({}, obj)` karta tha, ab `{ ...obj }` se ho jata hai.